Шпаргалка к тестированию

Большие данные, ИИ, программирование и математика

1. Математика

1.1. Многочлены и формулы Виета

Квадратное уравнение:

$$x^2 + px + q = 0$$

Если его корни x_1, x_2, то:

$$x_1 + x_2 = -p, \qquad x_1 x_2 = q$$

Обозначения:

$$S = x_1 + x_2, \qquad P = x_1 x_2$$

Полезные формулы:

$$x_1^2 + x_2^2 = (x_1 + x_2)^2 - 2x_1x_2 = S^2 - 2P$$
$$x_1^3 + x_2^3 = (x_1 + x_2)^3 - 3x_1x_2(x_1 + x_2) = S^3 - 3PS$$
$$\frac{1}{x_1} + \frac{1}{x_2} = \frac{x_1 + x_2}{x_1 x_2} = \frac{S}{P}$$

Если квадратное уравнение имеет вид:

$$ax^2 + bx + c = 0,$$

то:

$$x_1 + x_2 = -\frac{b}{a}, \qquad x_1 x_2 = \frac{c}{a}$$

Сумма коэффициентов многочлена $P(x)$:

$$P(1)$$

1.2. Остатки и сравнения по модулю

Если число a делится на m с остатком r, то:

$$a = mq + r, \qquad 0 \le r < m$$

Сравнение по модулю:

$$a \equiv b \pmod{m}$$

означает, что $a - b$ делится на m.

Свойства:

$$a \equiv b \pmod{m}, \quad c \equiv d \pmod{m}$$

тогда:

$$a + c \equiv b + d \pmod{m}$$
$$a - c \equiv b - d \pmod{m}$$
$$ac \equiv bd \pmod{m}$$
$$a^k \equiv b^k \pmod{m}$$

Последняя цифра числа — это остаток по модулю 10.

Циклы последних цифр:

- 2^n: $2, 4, 8, 6$ (цикл длины 4)
- 3^n: $3, 9, 7, 1$ (цикл длины 4)
- 7^n: $7, 9, 3, 1$ (цикл длины 4)
- 9^n: $9, 1$ (цикл длины 2)

1.3. Суммы и прогрессии

Сумма первых n натуральных чисел:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Сумма первых n нечётных чисел:

$$1 + 3 + 5 + \cdots + (2n-1) = n^2$$

Арифметическая прогрессия:

$$a_1,\ a_1 + d,\ a_1 + 2d,\ \ldots$$

Сумма первых n членов:

$$S_n = \frac{n(a_1 + a_n)}{2}$$

Количество членов прогрессии:

$$n = \frac{a_n - a_1}{d} + 1$$

1.4. НОД специальных выражений

$$\gcd(a^m - 1,\ a^n - 1) = a^{\gcd(m,n)} - 1$$

Если степени подходят по условию задачи с плюсом, иногда используется:

$$\gcd(a^m + 1,\ a^n + 1) = a^{\gcd(m,n)} + 1$$

но это надо применять аккуратно, когда это действительно подходит по условию.

1.5. Телескопические суммы

Основое разложение:

$$\frac{1}{n(n+1)} = \frac{1}{n} - \frac{1}{n+1}$$

Поэтому суммы вида

$$\frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \cdots + \frac{1}{n(n+1)}$$

удобно сворачиваются.

1.6. Комбинаторика

Правило суммы: если объект можно выбрать одним из нескольких способов, то общее число вариантов равно сумме.

Правило произведения: если действие состоит из нескольких шагов, то число вариантов равно произведению числа вариантов на каждом шаге.

Факториал:

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

Перестановки из n различных элементов:

$$n!$$

Перестановки с повторениями:

$$\frac{n!}{k_1! k_2! \dots}$$

Сочетания:

$$C_n^k = \frac{n!}{k!(n-k)!}$$

Размещения:

$$A_n^k = \frac{n!}{(n-k)!}$$

Число способов распределить n одинаковых объектов между k людьми:

$$x_1 + x_2 + \dots + x_k = n$$

$$\# = C_{n+k-1}^{k-1}$$

1.7. Вероятность

Классическая вероятность:

$$P = \frac{\text{число благоприятных исходов}}{\text{число всех исходов}}$$

Для независимых событий:

$$P(A \cap B) = P(A) \cdot P(B)$$

Если бросают два кубика, то всего исходов:

$$6 \cdot 6 = 36$$

1.8. Бинарные строки

Количество строк длины n из символов 0 и 1:

$$2^n$$

Если запрещены два нуля подряд, часто возникает рекурсия Фибоначчи:

$$a_n = a_{n-1} + a_{n-2}$$

2. Программирование: Python

2.1. Базовые конструкции

```
for i in range(n):
    ...

while condition:
    ...

if condition:
    ...
elif other_condition:
    ...
else:
    ...
```

2.2. Строки

Перебор символов:

```
for c in s:
    ...
```

Полезные методы:

```
s.lower()      # нижний регистр
s.split()      # разбить по пробелам
c.isalpha()    # буква?
c.isdigit()    # цифра?
c.isspace()    # пробельный символ?
```

Извлечение слов из текста:

```
import re
words = re.findall(r"[a-z]+", text.lower())
```

2.3. Словари и частоты

Частотный словарь:

```
freq = {}
for x in data:
    freq[x] = freq.get(x, 0) + 1
```

Элемент с максимальной частотой:

```
answer = max(freq, key=freq.get)
```

Сортировка по частоте:

```
result = sorted(freq.items(), key=lambda x: x[1], reverse=True)
```

2.4. Работа с файлами

Чтение всего файла:

```
with open("file.txt", "r") as f:
    text = f.read()
```

Чтение построчно:

```
with open("file.txt", "r") as f:
    for line in f:
        line = line.strip()
```

Чтение чисел:

```
with open("numbers.txt", "r") as f:
    numbers = list(map(int, f.read().split()))
```

Чтение JSON-строк:

```
import json
with open("events.txt", "r") as f:
    for line in f:
        event = json.loads(line)
```

2.5. Графы

Если граф задан матрицей смежности, то степень вершины равна сумме элементов строки:

```
degree = sum(graph[i])
```

Поиск в ширину (BFS) для кратчайших путей в невзвешенном графе:

```
from collections import deque

def bfs(graph, start):
    n = len(graph)
    dist = [-1] * n
    q = deque([start])
    dist[start] = 0
    while q:
        v = q.popleft()
        for u in range(n):
            if graph[v][u] == 1 and dist[u] == -1:
                dist[u] = dist[v] + 1
                q.append(u)
    return dist
```

DFS для компонент связности:

```
def dfs(graph, v, visited):
    visited[v] = True
    for u in range(len(graph)):
        if graph[v][u] == 1 and not visited[u]:
            dfs(graph, u, visited)
```

2.6. Алгоритмы для последовательностей

Слияние двух отсортированных списков:

```
def merge(a, b):
    i = j = 0
    res = []
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            res.append(a[i])
            i += 1
        else:
            res.append(b[j])
            j += 1
    res += a[i:]
    res += b[j:]
    return res
```

Максимальная сумма подпоследовательности без соседних элементов:

```
dp = [0] * len(a)
dp[0] = a[0]
dp[1] = max(a[0], a[1])
for i in range(2, len(a)):
    dp[i] = max(dp[i-1], dp[i-2] + a[i])
print(dp[-1])
```

Баланс в бинарной строке:

```
balance = 0
count = 0
for c in s:
    if c == '1':
        balance += 1
    else:
        balance -= 1
    if balance == 0:
        count += 1
print(count)
```

2.7. Асимптотика

Если на каждом шаге число делится на 2, 3 и т.п., то сложность обычно:

$$O(\log n)$$

Если один цикл по n элементам:

$$O(n)$$

Если два вложенных цикла по n:

$$O(n^2)$$

3. Анализ данных

3.1. Таблицы и CSV

CSV — текстовый формат таблиц, где значения в строке разделены запятыми.
Обычно в задачах используются таблицы: строки — объекты, столбцы — признаки.

3.2. Pandas: самое нужное

Подключение:

```
import pandas as pd
```

Чтение CSV:

```
df = pd.read_csv("data.csv")
```

Первые строки:

```
df.head()
```

Количество строк:

```
len(df)
```

Выбор столбцов:

```
df[["user_id", "age"]]
```

Фильтрация строк:

```
df[df["age"] > 18]
```

Уникальные значения:

```
df["user_id"].nunique()
```

Группировка:

```
df.groupby("user_id").size()
```

Агрегация:

```
df.groupby("user_id")["duration"].mean()
```

Сортировка:

```
df.sort_values("age")
df.sort_values("age", ascending=False)
```

Объединение таблиц:

```
pd.merge(table1, table2, on="user_id")
```

Проверка пропусков:

```
df["birth_date"].isna().sum()
```

Проверка принадлежности:

```
df["user_id"].isin(active_users)
```

Размер таблицы:

```
df.shape
df.shape[0]   # число строк
df.shape[1]   # число столбцов
```

3.3. Даты

Преобразование в дату:

```
df["date"] = pd.to_datetime(df["date"])
```

Фильтр по диапазону дат:

```
march = df[(df["date"] >= "2023-03-01") &
           (df["date"] <  "2023-04-01")]
```

3.4. Строковые операции в pandas

Разбиение строки:

```
df["action_list"] = df["action"].str.split(", ")
```

Разворачивание списков в строки:

```
df = df.explode("action_list")
```

3.5. Базовая статистика

Среднее:

```
df["value"].mean()
```

Медиана:

```
df["value"].median()
```

Корреляция:

```
df["x"].corr(df["y"])
```

3.6. Визуализация

Диаграмма рассеяния:

```
import matplotlib.pyplot as plt
plt.scatter(df["x"], df["y"])
plt.show()
```

Линейный график:

```
plt.plot(x, y)
plt.show()
```

4. Машинное обучение и ИИ

4.1. Основные типы задач

- Регрессия — предсказание числа.
- Классификация — предсказание класса.

Объект описывается признаками:

$$x = (x_1, x_2, \ldots, x_n)$$

Модель строит предсказание:

$$\hat{y} = f(x)$$

4.2. Линейная регрессия

Формула модели:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

Если один признак:

$$\hat{y} = w_0 + w_1 x$$

Пример на Python:

```
y_pred = 2 + 3*x1 - 4*x2
```

4.3. Логистическая регрессия

Сначала вычисляется:

$$z = w_0 + w_1 x_1 + w_2 x_2$$

Потом сигмоида:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Значение $\sigma(z)$ лежит в интервале $(0, 1)$ и интерпретируется как вероятность.

Если

$$\sigma(z) > p,$$

то объект относят к классу 1, иначе к классу 0.

Код:

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

4.4. Метрики качества

Среднеквадратичная ошибка:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Чем меньше MSE, тем лучше.

Код:

```
mse = ((y_true - y_pred) ** 2).mean()
```

Accuracy:

$$accuracy = \frac{\text{число правильных ответов}}{\text{общее число объектов}}$$

Код:

```
accuracy = (y_pred == y_true).mean()
```

Коэффициент детерминации:

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Диапазон:

$$(-\infty, 1]$$

Чем ближе R^2 к 1, тем лучше.

4.5. Подбор параметров

Идея: перебрать параметры и выбрать те, где метрика лучше.

```
best_r2 = -1
for w0 in range(-5, 5):
    for w1 in range(-5, 5):
        y_pred = w0 + w1 * x
        r2 = compute_r2(y, y_pred)
        best_r2 = max(best_r2, r2)
```

4.6. Кусочные модели

Кусочно-константная модель — на каждом интервале своё постоянное значение.

Кусочно-линейная модель — на каждом интервале своя линейная регрессия.

5. Робототехника

5.1. RGB-светодиод

Если одновременно включены красный и зелёный каналы, получится жёлтый цвет.

5.2. Мощность и время работы

Мощность:

$$P = U \cdot I$$

Если есть несколько потребителей, суммарная мощность:

$$P_{\text{общ}} = P_1 + P_2 + \dots$$

Время работы от аккумулятора:

$$t = \frac{E}{P}$$

где E — энергия аккумулятора в Вт·ч, P — мощность в Вт.

5.3. АЦП Arduino

Если:

$$analogRead(A0) = N,$$

то напряжение:

$$V = \frac{N}{1023} \cdot 5$$

5.4. Делитель напряжения

Формула делителя:

$$V = V_{\text{in}} \cdot \frac{R_2}{R_1 + R_2}$$

Если известны V, V_{in} и один из резисторов, можно найти второй.

5.5. Момент силы

$$M = F \cdot r$$

где $F = mg$, r — плечо силы.